Author: Brendan Bianchi

SSD Performance Profiling – Project Report

AI Usage Statement

Claude was used for code creation, plot generation, assistance in data analysis, and writing support.

Abstract

This project characterizes a Samsung 990 EVO Plus 4 TB NVMe SSD on an AMD Ryzen 5 3600 desktop running Windows 11. It measures zero-queue latency, the throughput–latency trade-off as queue depth grows, block-size and read/write-mix effects, tail latency, and how device state changes performance. I built a reproducible harness around fio 3.43 (the `windowsaio` engine with unbuffered I/O) in MSYS2, with two g++ tools: an independent QD=1 reference benchmark and a helper that reads the volume's I/O counters, drive temperature and device facts. The full run took 4.5 hours over a 64 GiB test file on a dedicated NTFS partition. At QD=1, a 4 KiB random read took 99 µs and a 4 KiB random write 33 µs, because writes are acknowledged from the drive's cache. Random 4 KiB reads scaled almost linearly to a knee at a total queue depth of 32 (205k IOPS, 146 µs) and plateaued at about 255k IOPS. Little's Law showed that a single submitting thread kept only about 60% of its configured queue depth in flight, so single-thread results above QD 16 are host-bound. The drive's PCIe link trained at Gen3 ×4, capping the interface at 3.94 GB/s raw; the best observed throughput was 3.35 GB/s while writing into the SLC cache. After about 25.6 GB the cache was full and sustained sequential writes fell to roughly 67 MB/s with periodic recovery bursts, and 4 KiB random writes alternated between about 123k-IOPS bursts and garbage-collection stalls, ending at a median of 39k IOPS. Sequential-read throughput halved, from about 2.0 to 1.0 GB/s, after heavy 4 KiB random writing, because the drive's write history determines how its data is laid out on flash. Four misleading setups (the OS file cache, a synchronous engine, never-written blocks and NTFS zero-fill) inflated or distorted results by 2.5–65×, and each was exposed by the volume's byte counters or by Little's Law.

1. Introduction and goals

An NVMe SSD is a parallel machine: its controller spreads requests over many flash dies, so it delivers its rated performance only when many requests are outstanding at once. A single request, however, still pays the full cost of a flash read (tens of microseconds) or of the controller's write path. Storage therefore shows the same throughput–latency trade-off as memory: raising the queue depth fills idle dies and increases throughput until a saturation knee, after which more requests only wait longer. On top of this, flash cannot be overwritten in place, so performance also depends on what was written before: how full the drive's fast SLC write cache is, how much garbage collection is pending, and how the data is laid out.

This project measures these effects on the SSD in my own desktop. It asks five questions: what a single unqueued read or write costs; how block size, access pattern and read/write mix change throughput and latency; where the queue-depth knee lies and whether Little's Law explains the curve; how the latency tail behaves near the knee; and how device state (cache exhaustion, garbage collection, write history) changes the numbers. As in the previous projects, the goal is also measurement discipline: controlling

the environment, validating that each measurement means what it claims, reporting uncertainty, and explaining results that don't match expectations.

The report first states my hypotheses, then describes the setup and validation. Sections 5–11 present the experiments, Section 12 covers misleading setups, Section 13 lists anomalies and limitations, and Section 14 concludes.

Required experiments and where each is answered:

Assignment requirement	Section	Figure(s)
Zero-queue baselines: QD=1 latency, 4 KiB random and 128 KiB sequential, read and write, with p95/p99	5	Fig. 1
Block-size sweep: throughput and latency from the same runs	6	Fig. 2
Read/write mix sweep (100/0, 70/30, 50/50, 0/100)	7	Fig. 3
Queue-depth/parallelism sweep, knee, Little's Law	8	Fig. 4, 5
Maximum small-block IOPS and large-block throughput	9	—
Tail latency at mid-QD and near the knee	10	Fig. 6
Working-set/LBA range and burst vs steady state	11	Fig. 7, 8, 9
Misleading setup (negative/failure case)	12	Fig. 10
Correctness validation	4	—
Honest anomalies	13	—

2. Hypotheses (written before the experiments)

These predictions were committed to the repository (`docs/REPORT.md`) on Oct 21, two days before the measurement run, so the results could be judged against them rather than explained after the fact. The table summarizes each prediction and its outcome; the reasoning behind each follows.

ID	Prediction	Measured	Verdict
H1	QD=1: 4K read 50–100 µs; 4K write 10–30 µs (below read); 128K read 100–200 µs; 128K write 40–100 µs	99 / 33 / 142 µs; 128K write median 68 µs but mean 213 µs	Supported (128K write mean inflated by its tail)
H2	Throughput rises with block size and saturates at 64–128 KiB; random ≈ sequential at ≥ 64–128 KiB	Sequential read saturates at 32 KiB (≈ 1.95 GB/s); random read converges only at 256 KiB; writes flat at 0.4–0.6 GB/s	Partly

Author: Brendan Bianchi

ID	Prediction	Measured	Verdict
H3	Mixed IOPS below linear interpolation; read p99 rises with write share	70/30: 65.4k vs 74.8k interpolated (−13%); read p99 447 → 1,379 µs	Supported
H4	Knee at QD 16–64; latency near QD=1 below the knee; one thread CPU-bound at 150–400k IOPS	Knee QD 32 (205k IOPS, 146 µs); 128K read obeys W = QD/X to 0.2%; one thread capped at 84k IOPS	Mostly supported (single-thread cap lower)
H5	Tail ratios p99/p50 and p99.9/p50 grow toward the knee; 70:30 mix adds a millisecond-scale read tail	p99/p50 1.9 → 2.8 but p99.9/p50 flat at about 4; 70:30 read p99.9 5.3 ms	Partly
H6	DRAM-less (HMB) drive: faster on small spans, slower on large	Flat from 1 to 32 GiB; full 64 GiB span +10% latency at QD=1 only	Partly (small effect)
H7	Random-write steady state 20–50% of burst; SLC cliff after tens–hundreds of GB; throttling only near 70–80 °C	Final median 39k vs burst 123k IOPS (32%); cliff after 25.6 GB; max 54 °C, no throttling	Supported
H8	Buffered reads > 500k IOPS at 1–3 µs with ~0 device bytes; sync engine at iodepth 32 behaves as QD=1; 32 × 4 jobs gives ~128 in flight; unwritten reads impossibly fast	150k IOPS at 6 µs, 0.00 device bytes; L = 0.99; L = 88; 210k vs 82k IOPS	Mostly supported (buffered rate lower)
H9	Random writes into an extended-but-unwritten NTFS file cause hidden zero-fill writes and very long stalls	Device wrote 65× fio's bytes; worst write 311–500 ms	Supported

H1 – QD=1 latency. I expected a 4 KiB random read to take 50–100 µs, dominated by the flash array read (roughly 40–80 µs for TLC) plus about 10 µs of controller, PCIe and host-stack time. A 4 KiB write should be faster (10–30 µs), because the drive acknowledges it once the data is in its write buffer and programs the flash later. A 128 KiB sequential read should take 100–200 µs and a 128 KiB write 40–100 µs. Writes should have the heavier p99.9, from buffer flushes and garbage collection.

H2 – Block size. Larger blocks amortize per-command overhead, so throughput should rise with block size until the flash parallelism or the link limits it, around 64–128 KiB for sequential access. IOPS should fall with block size, and latency should stay nearly flat for small blocks and grow linearly once transfer time dominates. A large random I/O is striped across many dies anyway, so random should approach sequential at 64–128 KiB.

H3 – Read/write mix. Reads that land on a die busy programming or erasing must wait, so mixed workloads should deliver fewer IOPS than a linear interpolation between pure reads and pure writes, and the read tail should grow with the write share.

H4 – Queue depth. IOPS should grow almost linearly with queue depth while idle dies remain, with latency near the QD=1 value. Past the knee (expected at QD 16–64), throughput should plateau, and by Little's Law the mean latency should grow in proportion to the queue depth. A single submitting thread might become CPU-bound before the drive saturates, so a four-thread series should reach a higher plateau.

H5 – Tail latency. Queueing turns variation in service time into variation in waiting time, so the ratios of p99 and p99.9 to the median should grow toward the knee. A 70:30 mix at the knee should add a millisecond-scale read tail from program and erase interference.

H6 – Working set. A drive with on-board DRAM holds its whole mapping table and should show flat random-read latency across spans. A DRAM-less drive, which caches only part of its mapping table in host memory (HMB), should be faster on small spans and slower on large ones. (The datasheet later confirmed this drive is DRAM-less.)

H7 – Device state. Random-write IOPS should start high and decay to a steady state of 20–50% of the burst once garbage collection must run. Sustained sequential writes should drop sharply after tens to hundreds of gigabytes, when the SLC cache is full. Thermal throttling should occur only if the drive approaches 70–80 °C.

H8 – Misleading setups. Buffered random reads on a working set that fits in RAM should report more than 500k IOPS at 1–3 µs, served by the file cache with almost no device reads. A synchronous engine with `iodepth=32` should behave exactly like QD=1.

H9 – NTFS valid data length. When a file's end is set without writing data, NTFS must fill the gap with zeros before a later write beyond it completes. Random writes into such a file should therefore cause many times more device writes than fio issues, and very long individual writes.

The ranges reflect typical published figures for TLC NVMe drives. The results that departed most from them were the single-thread IOPS limit, the flat p99.9 ratio and the very low post-cache write rate; all three are discussed in the results.

3. Experimental setup and methodology

3.1 System configuration

Item	Value
SSD	Samsung SSD 990 EVO Plus 4 TB (MZ-V9S4T0), firmware 2B2QKXG7; TLC V-NAND, DRAM-less with Host Memory Buffer (HMB); rated 7,250 MB/s read, 6,300 MB/s write, 1,050K random-read IOPS on PCIe 4.0 ×4
Sectors and cache	512 B logical / 4,096 B physical; device write cache enabled; TRIM enabled
PCIe link	Trained at 8 GT/s (Gen3) ×4; the drive supports up to 32 GT/s. Raw ceiling 8 × 4 × 128/130 / 8 = 3.94 GB/s per direction

Item	Value
Drive role	System disk (holds Windows C:), 5 partitions
Test target	64 GiB file `D:\ssdtest\fio-scratch.bin` on a dedicated NTFS partition (4 KiB clusters) of the same SSD; 1.7% of drive capacity
CPU	AMD Ryzen 5 3600, 6 cores / 12 threads, SMT on, 3.59 GHz nominal
Memory	32 GB
OS	Windows 11 Pro, build 26200
fio	3.43 (official Windows build), `ioengine=windowsaio` (I/O completion ports), `direct=1`
Toolchain	MSYS2 UCRT64, g++ 16.2.0 (`-O2 -std=c++17 -static`); Python 3.14 with numpy, pandas, matplotlib
Power	Balanced plan on AC: minimum processor state 5%, maximum 100%
Pinning	fio `cpus_allowed=2-5`; reference tool pinned to CPU 2
Shell	MSYS2 UCRT64, run as administrator
Temperature	38 °C at start, 39–54 °C during the run; drive warning threshold 81 °C
Repetitions	3 per point, each 5 s warm-up + 30 s measured (tail runs 90 s); repetition loop outermost

3.2 Target and safety

All tests ran against a 64 GiB file on a separate NTFS partition created by shrinking C:, so no existing data could be overwritten. The harness refuses any file whose name lacks `fio-scratch` or that it did not create itself, and refuses network, RAM and FAT32 volumes. The partition sits on the same physical SSD as Windows, so the OS's own I/O still reaches the drive; I/O on C: is, however, excluded from the byte counters, which are read from the test volume only (Section 3.4).

3.3 fio configuration

Every measurement uses unbuffered I/O (`direct=1`, which fio maps to `FILE_FLAG_NO_BUFFERING`), so the Windows file cache neither serves reads nor absorbs writes. The `windowsaio` engine uses I/O completion ports, so `iodepth` is the true number of outstanding requests per job; the effective queue depth is `iodepth × numjobs`, and all plots use that total. Sequential runs with several jobs give each job its own disjoint stripe, so the drive sees separate streams rather than one stream read several times. Data is incompressible (`refill_buffers`, `buffer_compress_percentage=0`), random offsets are aligned to the block size and drawn afresh each run, and latency is fio's total latency from submission to completion. For each run the harness writes a fully resolved job file next to its results, so every number can be traced to its exact configuration.

Preconditioning. Before any measurement, the file was written sequentially twice end to end (128 KiB, QD 32), then overwritten with 64 GiB of random 4 KiB writes. The first step guarantees that every block is mapped (a never-written block is answered without reading flash; Section 12) and that NTFS's valid-data length reaches the end of the file. The second fragments the mapping and wakes garbage collection, so write tests do not start from a fresh-drive state. The order of experiments was fixed: validation, the read/write sweeps, the misleading-setup demonstrations, and the steady-state test last, because it leaves the drive the most fragmented.

3.4 Observability

A small g++ helper (`winstat.exe`) reads cumulative read/write byte counts for the test volume with `IOCTL_DISK_PERFORMANCE` before and after each run, plus the drive's temperature, model, firmware, write-cache state and partition layout. A PowerShell script records the OS build, CPU, power plan and the PCIe link speed and width. Comparing the volume's byte counts with the bytes fio reports shows whether I/O really reached the drive, and Little's Law (IOPS × mean latency) shows how many requests were really in flight.

3.5 Statistics and uncertainty

Every point was repeated three times, with the repetition loop outermost, so slow drift (temperature, garbage collection) affects all points equally rather than biasing one. Tables report the mean and, where shown, a 95% confidence interval from Student's t-distribution ($t = 4.30$ for three samples). With three samples the interval is wide, so the figures show the full range of the three repetitions as whiskers. Write results varied far more between repetitions than reads; that variation is a property of the drive's internal state, discussed in Section 11, rather than measurement noise.

The knee is defined as the queue depth that maximizes Kleinrock's power, throughput divided by mean latency; a second, marginal criterion marks the last queue depth before doubling it gave less than 10% more throughput for at least 50% more latency.

4. Correctness validation

The data path is correct and fio's numbers agree with an independent tool, but Little's Law shows that deep queues of small blocks from one thread are host-bound. Six automated checks produced 627 check results across all runs:

Check	What it tests	Result
V1 crc32c write + verify (256 MiB)	Wrong offsets, corrupted data path	Pass, 0 errors
V2 fio vs reference tool at QD=1	fio configuration and engine overhead	Reads within 9%; writes differ 17–28% (8 of 12 flagged WARN)
V3 `dd` unbuffered 128 KiB read	Second bandwidth reference	fio 842 vs dd 881 MB/s (−4.3%), Pass
V4 Little's Law, IOPS × mean	Whether the queue was	220 Pass, 14 Warn, 58 Fail; every Fail is

Check	What it tests	Result
latency ≈ configured QD	really full	a 4–16 KiB run at QD ≥ 16
V5 volume byte counters ≈ fio bytes	File-cache hits, hidden writes	287 Pass, 4 Fail (bursty writes, see Section 13)
V6 throughput ≤ PCIe raw ceiling	Impossible numbers	Pass: highest sweep result 2.49 GB/s vs 3.94 GB/s

A further 14 V4 and 15 V5 failures from the misleading-setup experiment are expected and labeled as such; they are the evidence that experiment relies on (Section 12).

Reference implementation. `ref_qd1.exe` is about 200 lines of g++ with no fio code: one thread opens the target with `FILE_FLAG_NO_BUFFERING`, issues one synchronous read or write at a time and times each with `std::chrono::steady_clock`.

Case (QD=1)	fio mean / IOPS	Reference mean / IOPS	Difference in latency
4 KiB random read	101.3 µs / 9,816	110.9 µs / 9,023	−8.7%
128 KiB sequential read	158.0 µs / 6,427	170.9 µs / 5,969	−7.6%
4 KiB random write	34.2 µs / 28,368	26.7 µs / 37,343	+28.0%
128 KiB sequential write	192.9 µs / 5,060	164.4 µs / 6,211	+17.3%

For reads the two tools agree within 9%, which confirms that fio's QD=1 read latency is the drive's, not an artifact of the engine. For writes, fio reports latencies 7.5 µs (4 KiB) and 28 µs (128 KiB) higher in all three repetitions. Both tools issue the same unbuffered write, so the gap must come from the host side; the likely cause is fio's completion path, discussed as anomaly A5. Write results in this report are therefore upper bounds on the drive's own write latency, by up to about 7 µs at 4 KiB.

Little's Law as a check. At QD=1, IOPS × mean latency averaged 0.986 across all runs, so fio kept exactly one request in flight. Every run with blocks of 32 KiB or more matched its configured queue depth within 3%. With 4 KiB blocks and one job, however, only 0.51–0.78 of the configured depth was actually outstanding at QD ≥ 16 (for example 20 of 32), and with four jobs 0.59–0.97. The latency values themselves are correct, but the requests that were not in flight were waiting inside fio for the job thread to resubmit them. The drive therefore saw a shallower queue than configured. Section 8 uses the measured in-flight count wherever this matters.

Byte accounting. In 287 of the 291 non-demonstration runs the volume's counters matched fio's byte count after allowing for the 5-second warm-up. That rules out file-cache hits and hidden writes for all measurements reported in Sections 5–11.

5. Zero-queue latency (QD=1)

One job, one request outstanding at a time, so latency is pure service time with no queueing. Mean ± 95% CI over three repetitions; percentiles in µs:

Author: Brendan Bianchi

Case	IOPS	Throughput	Mean (µs)	p50	p95	p99	p99.9
4 KiB random read	10,014	41 MB/s	99.3 ± 17.6	88.2	160.8	179.2	376.8
4 KiB random write	29,571	121 MB/s	32.9 ± 6.0	27.8	39.3	50.6	73.6
128 KiB sequential read	7,139	936 MB/s	142.0 ± 64.3	123.6	265.6	324.3	568.0
128 KiB sequential write	4,599	603 MB/s	213.0 ± 48.8	67.8	86.9	434.5	29,753

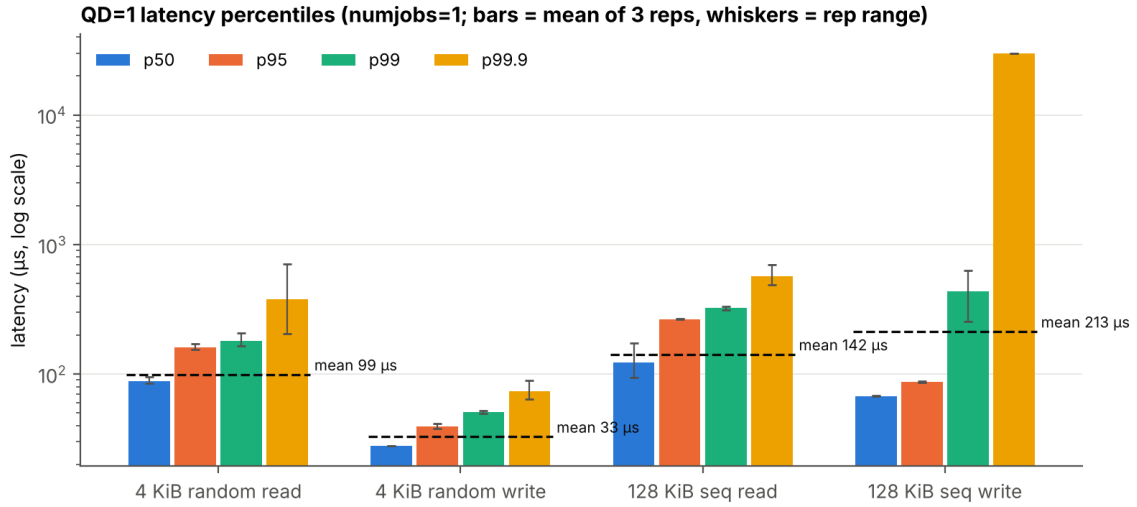


Figure 1: QD=1 latency percentiles; dashed lines mark the mean. Log scale.

A 4 KiB random read took a median of 88 µs. Almost all of that is the flash array read: a TLC page read takes several tens of microseconds, and on a DRAM-less drive the controller may first need a mapping-table entry from host memory. The PCIe transfer of 4 KiB at Gen3 speed takes about 1 µs, and the Windows I/O stack adds roughly 10 µs. A 4 KiB random write was three times faster (median 28 µs), because the drive acknowledges a write as soon as the data reaches its write buffer and programs the flash later. At QD=1, then, a small read costs three times as much as a small write.

A 128 KiB sequential read took a median of 124 µs, about 35 µs more than a 4 KiB read. Moving 128 KiB over a Gen3 ×4 link at 3.94 GB/s takes 33 µs, which accounts for almost all of the difference: the flash reads for the 32 pages run in parallel across dies, so their cost barely grows with size. Its confidence interval (±64 µs) is the widest among reads. The three repetitions measured 124, 172 and 130 µs, and Section 11 shows why: sequential-read speed depends on how the data was written.

The 128 KiB write is the clearest case of a heavy tail. Its median (68 µs) is close to the 33 µs transfer time plus the buffer acknowledgment, but about one write in a thousand waited roughly 30 ms (p99.9 = 29.8 ms), apparently while the drive emptied its write buffer into flash. Those rare stalls tripled the mean to 213 µs and made p99.9/p50 = 439, against 4.3 for 4 KiB reads. A mean alone would badly misdescribe this workload, which is why the report gives percentiles throughout. H1 is supported: all four medians fell inside the predicted ranges, and writes had the heavier p99.9 as expected.

Author: Brendan Bianchi

6. Block-size sweep

Queue depth 32, one job; throughput and mean latency come from the same runs. Each cell is MB/s with mean latency in parentheses (mean of three repetitions):

Block size	Sequential read	Random read	Sequential write	Random write
4 KiB	320 (0.25 ms)	343 (0.24 ms)	232 (0.31 ms)	193 (0.42 ms)
16 KiB	1,345 (0.27 ms)	1,265 (0.34 ms)	402 (1.2 ms)	431 (1.1 ms)
32 KiB	1,916 (0.54 ms)	1,047 (1.0 ms)	391 (3.0 ms)	440 (2.4 ms)
64 KiB	1,953 (1.1 ms)	1,207 (1.7 ms)	455 (4.7 ms)	433 (5.6 ms)
128 KiB	1,944 (2.2 ms)	1,583 (2.7 ms)	583 (7.2 ms)	406 (11.1 ms)
256 KiB	2,107 (4.0 ms)	1,922 (4.6 ms)	505 (16.9 ms)	501 (18.9 ms)
512 KiB	2,119 (8.0 ms)	—	483 (35.7 ms)	—
1 MiB	2,110 (16.1 ms)	—	439 (78.6 ms)	—

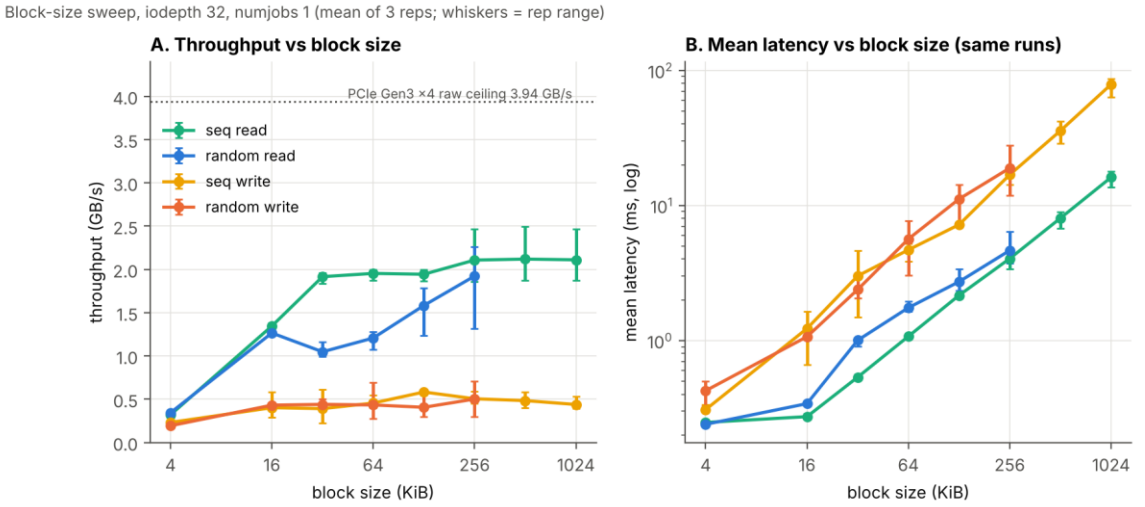


Figure 2: Throughput (A) and mean latency (B) against block size, from the same runs. Whiskers show the range of the three repetitions.

At 4 KiB every pattern delivered about 80k IOPS (320–343 MB/s for reads), the same ceiling as the single-thread queue-depth sweep in Section 8. That ceiling comes from the submitting thread, not the drive, so the 4 KiB points here understate the drive. From 16 KiB upward, sequential reads rose quickly and flattened at 32 KiB, near 1.95 GB/s, rising only to 2.1 GB/s by 1 MiB. That plateau is about half of the 3.94 GB/s link ceiling. The link is therefore not the limit; the flash layout left by earlier writes is (Section 11).

Once throughput is flat, latency must grow in proportion to block size, by Little's Law: with 32 requests of size b outstanding at throughput X, each takes $W = 32 \times b / X$. For 1 MiB sequential reads that predicts 32 × 1.048 MB / 2.11 GB/s = 15.9 ms against 16.1 ms measured, and for 1 MiB writes 76 ms against 79

ms. Panel B shows this as straight lines of slope one above 32 KiB: past that size, a bigger block buys no throughput and costs proportionally more latency.

Random reads approached sequential reads only at 256 KiB (1.92 vs 2.11 GB/s), later than the 64–128 KiB I predicted, and 32 KiB random reads were slower than 16 KiB ones (1.05 vs 1.27 GB/s). Both results fit a mapping that preconditioning fragmented at 4 KiB granularity: a large random read then turns into many scattered flash reads (anomaly A2).

Writes never followed the block size at all. Above 16 KiB they stayed between 0.39 and 0.58 GB/s, with repetitions differing by up to 2.7×, because they were limited by how fast the drive could empty its SLC cache into TLC flash rather than by request size. The 1 MiB sequential write's p99.9 reached 0.93 s. H2 is only partly supported: throughput did saturate around 32–64 KiB for reads, but not at the interface, and writes were bound by device state throughout.

7. Read/write mix

Random 4 KiB, queue depth 32, one job; only the read share changes.

Reads	Total IOPS	Linear interpolation	Harmonic model	Read mean / p99 (µs)	Write mean / p99 (µs)
100%	84,113	—	—	241 / 447	—
70%	65,423	74,855	71,654	330 / 1,379	375 / 11,399
50%	63,588	68,681	65,215	388 / 1,625	308 / 640
0%	53,249	—	—	—	342 / 665

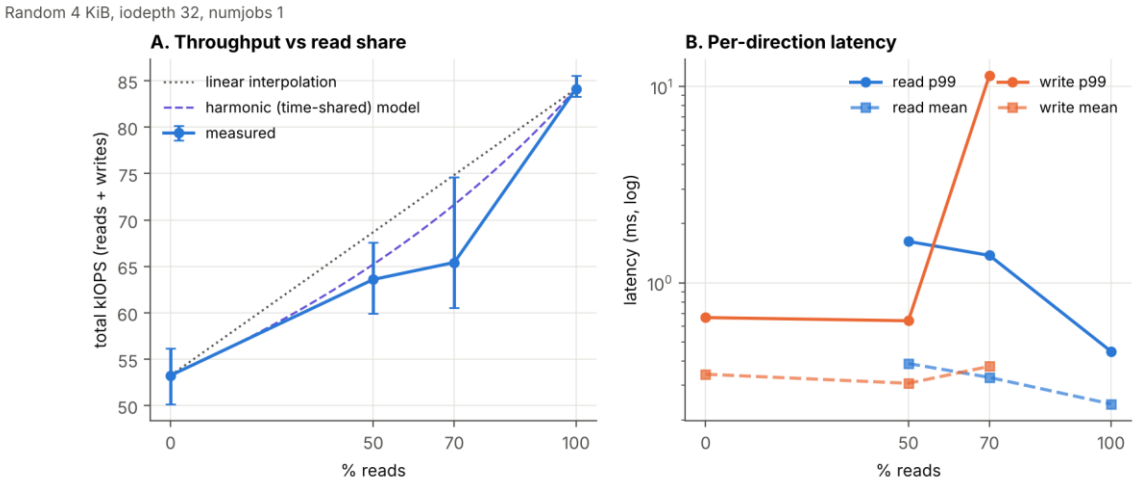


Figure 3: Total IOPS against read share, with linear and harmonic predictions (A), and per-direction latency (B). Whiskers show the range of three repetitions.

Mixing reads and writes cost throughput beyond simple averaging. The harmonic model assumes the drive simply time-shares between a read rate of 84.1k and a write rate of 53.2k IOPS:

IOPS(mix) = 1 / [r / IOPS(read) + (1 − r) / IOPS(write)], where r is the read share.

At 50/50 the measurement fell 2.5% below that model, and at 70/30 8.7% below it (13% below linear interpolation). The extra loss comes from interference: a read that lands on a die busy programming must wait for the program to finish, which takes far longer than a read. The read tail shows this directly: read p99 tripled from 447 µs with pure reads to 1,379 µs at 70/30 and 1,625 µs at 50/50, while the read mean grew only 37–61%.

The 70/30 point varied most between repetitions (60.5k, 61.1k and 74.6k IOPS). In the first two, about 1% of writes stalled for 16–18 ms (write p99 17.7 and 16.1 ms); in the third, no such stalls occurred (write p99 444 µs) and throughput landed almost exactly on the linear interpolation. The difference lies in the drive's state when each repetition started, not in the workload; the effect is the same write-buffer flushing seen in Sections 5 and 11. H3 is supported: mixed throughput fell below interpolation, and the read tail grew with the write share.

8. Queue depth, knee and Little's Law

Little's Law ties the three quantities of every queue together: the average number of requests in the system equals the arrival rate times the time each spends there.

L = λ × W, that is, requests in flight = IOPS × mean latency.

Below saturation, adding requests raises throughput at nearly constant latency; past it, throughput is fixed and latency grows in proportion to the queue depth. Four series were swept over a total queue depth of 1–256:

Series	QD=1	Knee (power / marginal)	At the knee	Plateau	At QD 256
4 KiB random read, 4 jobs	97 µs	QD 32 / 64	205k IOPS, 146 µs, p99 343 µs	255k IOPS (1.05 GB/s)	250k IOPS, 621 µs
4 KiB random read, 1 job	99 µs	QD 8 / 16	68k IOPS, 108 µs	84k IOPS	84k IOPS, 1,571 µs
4 KiB random write, 4 jobs	46 µs (QD 4)	QD 4 / 4	86k IOPS, 46 µs	64–75k IOPS	64k IOPS, 3,781 µs, p99 21 ms
128 KiB sequential read, 1 job	181 µs	QD 1 / 2	723 MB/s, 181 µs	0.99–1.0 GB/s	1.00 GB/s, 33.6 ms

Author: Brendan Bianchi

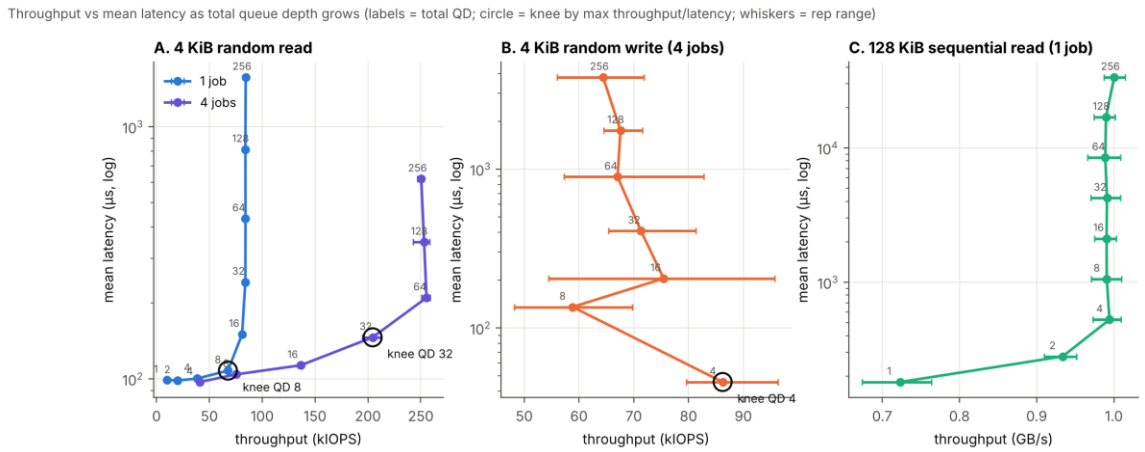


Figure 4: Throughput against mean latency as total queue depth grows; point labels are the total QD, circles mark the knee.

Random reads. With four jobs, IOPS grew almost linearly up to QD 16 (136k IOPS at 113 µs, against 97 µs at QD=4), because each added request found an idle flash die. The knee came at QD 32: 205k IOPS at 146 µs, where throughput per unit of latency peaked. Doubling to QD 64 bought 25% more IOPS for 44% more latency, and beyond that throughput stayed flat near 255k IOPS while latency tripled. The curve has the expected hockey-stick shape: a flat segment along which parallelism is free, then a vertical one along which it only adds waiting.

One thread versus four. A single job stopped at 84k IOPS from QD 32 onward, while four jobs reached 255k: the single-job ceiling belongs to the host. The job thread used 63% of a core in system time, and Little's Law confirms that it could not keep the queue full (Figure 5). At a configured QD of 32 only 20 requests were really in flight, and at 256 only 133. Requests that had completed were waiting inside fio to be reaped and resubmitted, so they appear in neither the drive's queue nor the measured latency.

Author: Brendan Bianchi

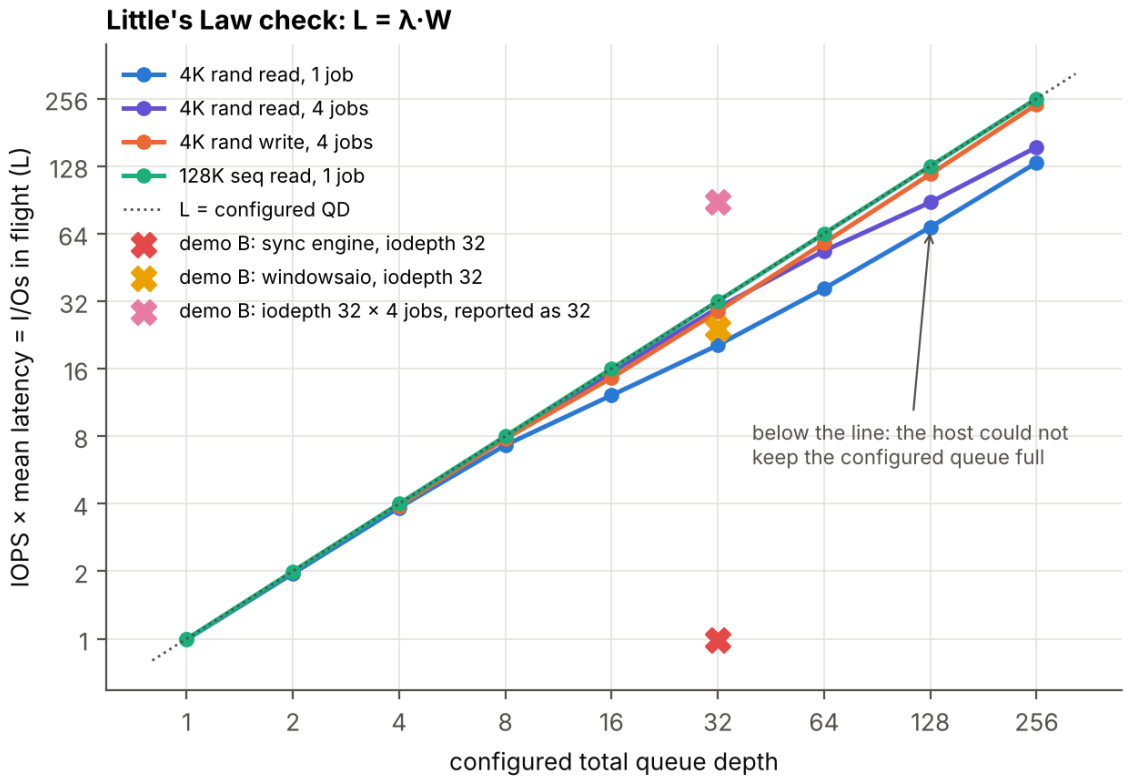


Figure 5: In-flight requests (IOPS × mean latency) against configured queue depth. The dotted diagonal is a full queue; the crosses are the misleading-setup demonstrations from Section 12.

Large sequential reads obey the law exactly. For 128 KiB reads, IOPS × latency matched the configured depth within 0.2% at every point, because each request carries so much work that the host is never the bottleneck. Once throughput saturated at 7,624 IOPS (1.0 GB/s, from QD 4 on), the law predicts W = 256 / 7,624 = 33.6 ms at QD 256; fio measured 33.56 ms. Above the knee, every doubling of queue depth doubled latency (526 µs at QD 4, 1.06 ms at 8, …, 33.6 ms at 256) without adding a byte per second. This is the clearest demonstration in the project that deep queues past saturation buy nothing.

Random writes. Writes had no clean knee. The best point was QD 4 (86k IOPS at 46 µs), and every deeper queue delivered less throughput: 59–75k IOPS with p99 between 2.6 and 21 ms. Writes are absorbed by the drive's buffer and drained in the background, so their throughput depends on how full the buffer is rather than on queue depth. The QD 4 runs came first in each series, right after the read series had given the drive time to drain; anomaly A4 discusses this ordering effect.

Where the plateau comes from. The random-read plateau of 255k IOPS (1.05 GB/s) is far below both the link ceiling and the drive's 1,050K-IOPS rating, which was measured on a faster PCIe 4.0 system. At QD 64 with four jobs, 54 of 64 requests (84%) were in flight, so the drive was close to its own limit there; at QD 128 and 256, only 69% and 61% were. The plateau is therefore partly host-bound, and 255k IOPS is a lower bound on what this drive can do with more submitting threads.

H4 is mostly supported: the knee fell at QD 32 inside the predicted 16–64 range, latency stayed near the QD=1 value below it, and the saturated region followed W = L/X. The single-thread limit (84k IOPS) was lower than the 150–400k I predicted for `windowsaio` on this CPU.

Author: Brendan Bianchi

9. Peak capability and the interface ceiling

The best sustained results were 251k IOPS for 4 KiB random reads and 2.49 GB/s for sequential reads; the only result near the link's capacity was a 3.35 GB/s write burst into the SLC cache.

Workload	Configuration	Result (mean ± 95% CI)	Share of 3.94 GB/s link
4 KiB random read	iodepth 64 × 4 jobs	251,122 ± 6,514 IOPS (1.03 GB/s), p99 1.4 ms	26%
4 KiB random write	iodepth 32 × 4 jobs	77,239 ± 46,246 IOPS (0.32 GB/s), p99 15.5 ms	8%
1 MiB sequential read	iodepth 16 × 2 jobs	1.04 ± 0.05 GB/s	26%
1 MiB sequential write	iodepth 16 × 2 jobs	0.38 ± 0.21 GB/s	10%
Best sequential read in any run	512 KiB, QD 32 (block-size sweep, rep 1)	2.49 GB/s	63%
Best write, per second	128 KiB, QD 32, into the SLC cache (Section 11)	3.35 GB/s	85%

The drive's PCIe link trained at 8 GT/s ×4 (Gen3) although the drive supports up to 32 GT/s. The datasheet confirms it is "backward compatible with PCIe 4.0 ×4 and 3.0 ×4", so this is the platform's limit, not a fault: a Ryzen 3000 CPU provides PCIe 4.0 only on X570 and B550 boards, and only in the CPU-attached M.2 slot. Gen3 ×4 moves 8 × 4 × 128/130 / 8 = 3.94 GB/s per direction before packet and NVMe protocol overhead, typically 3.4–3.6 GB/s usable. The 3.35 GB/s burst into the SLC cache is therefore effectively at the link limit: in its fastest state the drive could deliver more than this slot can carry, and its rated 7,250 MB/s read would need a Gen4 slot.

No other workload came near the link. Random 4 KiB reads at 251k IOPS used a quarter of it (a 4 KiB transfer limit would be about 960k IOPS), and Section 8 showed that this plateau is partly host-bound. The sustained sequential results were limited by device state: reads by the data layout left by earlier random writes (1.04 GB/s in the peak test against 2.49 GB/s earlier in the night, Section 11) and writes by the exhausted SLC cache. The rated figures (7,250 MB/s read, 6,300 MB/s write, 1,050K and 1,400K random IOPS) were measured on a Ryzen 9 7950X with a PCIe 4.0 X670E board and IOmeter, so they are not reachable on this platform and say little about sustained behavior. The large confidence intervals on the two write rows reflect the state dependence again: the four-job random-write repetitions measured 91k, 56k and 84k IOPS, and the 1 MiB write repetitions 446, 399 and 284 MB/s.

10. Tail latency at mid-QD and the knee

The tail runs were three times longer (90 s) and recorded fio's full latency histogram, so percentiles down to p99.99 rest on millions of samples. The mid and knee queue depths (8 and 32, four jobs) were taken automatically from the knee analysis of Section 8.

Author: Brendan Bianchi

4 KiB random reads	IOPS	p50	p95	p99	p99.9	p99.99	p99/p50	p99.9/p50
QD 1	8,974	99 µs	174 µs	193 µs	399 µs	2.12 ms	1.9	4.0
QD 8 (mid)	71,321	95 µs	186 µs	229 µs	377 µs	2.04 ms	2.4	4.0
QD 32 (knee)	195,727	133 µs	285 µs	369 µs	502 µs	2.13 ms	2.8	3.8
QD 32, 70:30 mix (reads)	131,666 total	153 µs	1.01 ms	1.43 ms	5.30 ms	16.6 ms	9.4	34.7

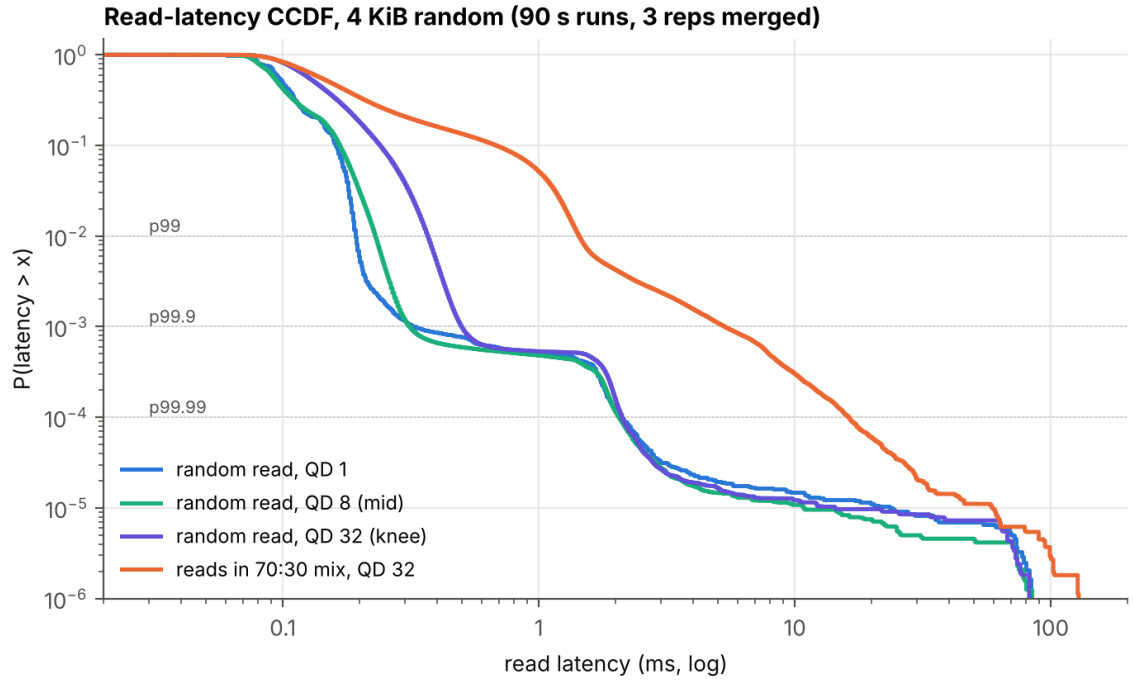


Figure 6: Complementary cumulative distribution of read latency: the fraction of reads slower than each latency. Dotted lines mark p99, p99.9 and p99.99.

Queueing moved the body of the distribution, not its far tail. From QD 1 to the knee, the median rose by a third and p99 nearly doubled (193 to 369 µs), as requests began to wait behind each other on busy dies. Beyond p99.9, however, the three pure-read curves in Figure 6 lie on top of each other: about 0.05% of reads took roughly 2 ms, and about one in 100,000 took 10–80 ms, at every queue depth. A tail that does not depend on load cannot come from queueing. It points to occasional events inside the drive or the host, such as background housekeeping, a mapping-table fetch from host memory or an interrupt-handling delay, that stall a read for a fixed time whatever else is outstanding. As a result p99.9/p50 stayed near 4 instead of growing as I predicted.

Writes changed the tail completely. With 30% writes at the same queue depth, 5% of reads took longer than 1 ms (against 0.05% for pure reads), p99.9 reached 5.3 ms and p99.99 16.6 ms. A read that arrives at a die busy programming or erasing must wait for that operation to finish, and these operations take far longer than a read.

This matters for any latency target. To keep p99 under 250 µs with pure reads, the queue-depth sweep allows at most QD 16 (136k IOPS, p99 247 µs); a 500 µs target allows QD 64 and the full 255k IOPS. With a 30% write share, even the knee misses a 1 ms p99 target. H5 is partly supported: p99/p50 grew toward the knee (1.9 to 2.8), and the mixed workload produced the predicted millisecond-scale read tail, but the extreme tail was set by load-independent events rather than queueing.

11. Working set and device state

11.1 Working set (LBA span)

4 KiB random reads over spans from 1 GiB to the full 64 GiB, all previously written:

Span	QD 1 mean / p99	QD 32 mean / p99	QD 32 IOPS
1 GiB	84.4 / 110.8 µs	240.9 / 437.6 µs	80,714
4 GiB	84.0 / 109.4 µs	238.3 / 426.7 µs	81,549
16 GiB	83.6 / 108.7 µs	239.0 / 429.4 µs	81,333
32 GiB	84.4 / 111.4 µs	238.0 / 429.4 µs	81,606
64 GiB (full)	93.2 / 164.2 µs	240.6 / 445.8 µs	84,098

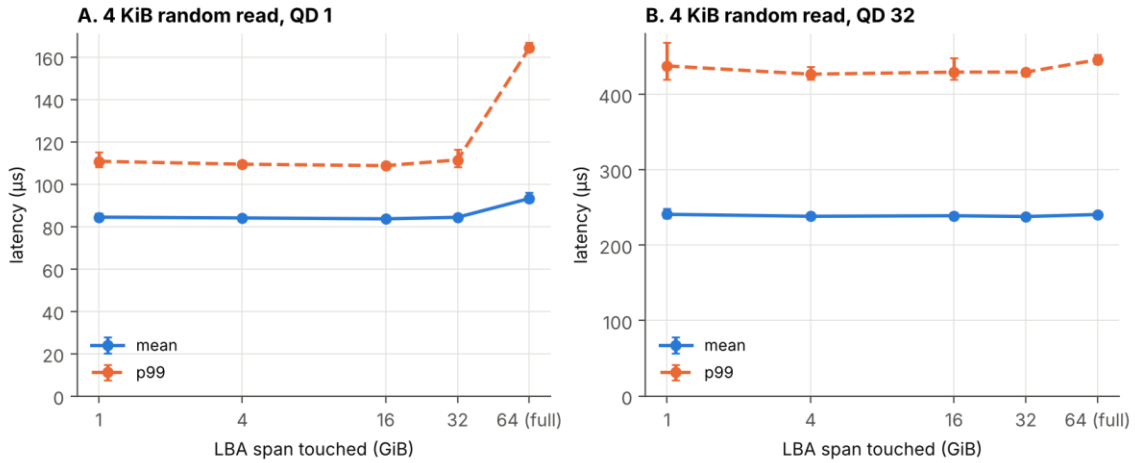


Figure 7: Mean and p99 latency of 4 KiB random reads against the span touched, at QD 1 (A) and QD 32 (B).

Latency was flat from 1 to 32 GiB and rose only at the full 64 GiB span, by 10% in the mean and 48% at p99, and only at QD 1. This drive has no DRAM and keeps the part of its logical-to-physical mapping table it needs in a Host Memory Buffer in system RAM. A read whose mapping entry is not cached needs an extra lookup before the data read. The pattern fits a mapping cache that covers somewhat more than 32 GiB of this file: inside that span every lookup hits, while the full span causes occasional misses. Windows does not report the HMB size, so this is an inference rather than a measurement. At QD 32 the extra lookups are hidden by the other 31 requests in flight. The full-span point also ran last in each

repetition, so a time effect cannot be fully excluded. H6 is partly supported: the predicted DRAM-less penalty exists, but it is small and confined to the largest span at QD 1.

11.2 Burst versus steady state

After a 5-minute idle, the drive took 600 s of 128 KiB sequential writes and then 30 minutes of 4 KiB random writes at a total QD of 128, with per-second logging and temperature sampled every 2 s.

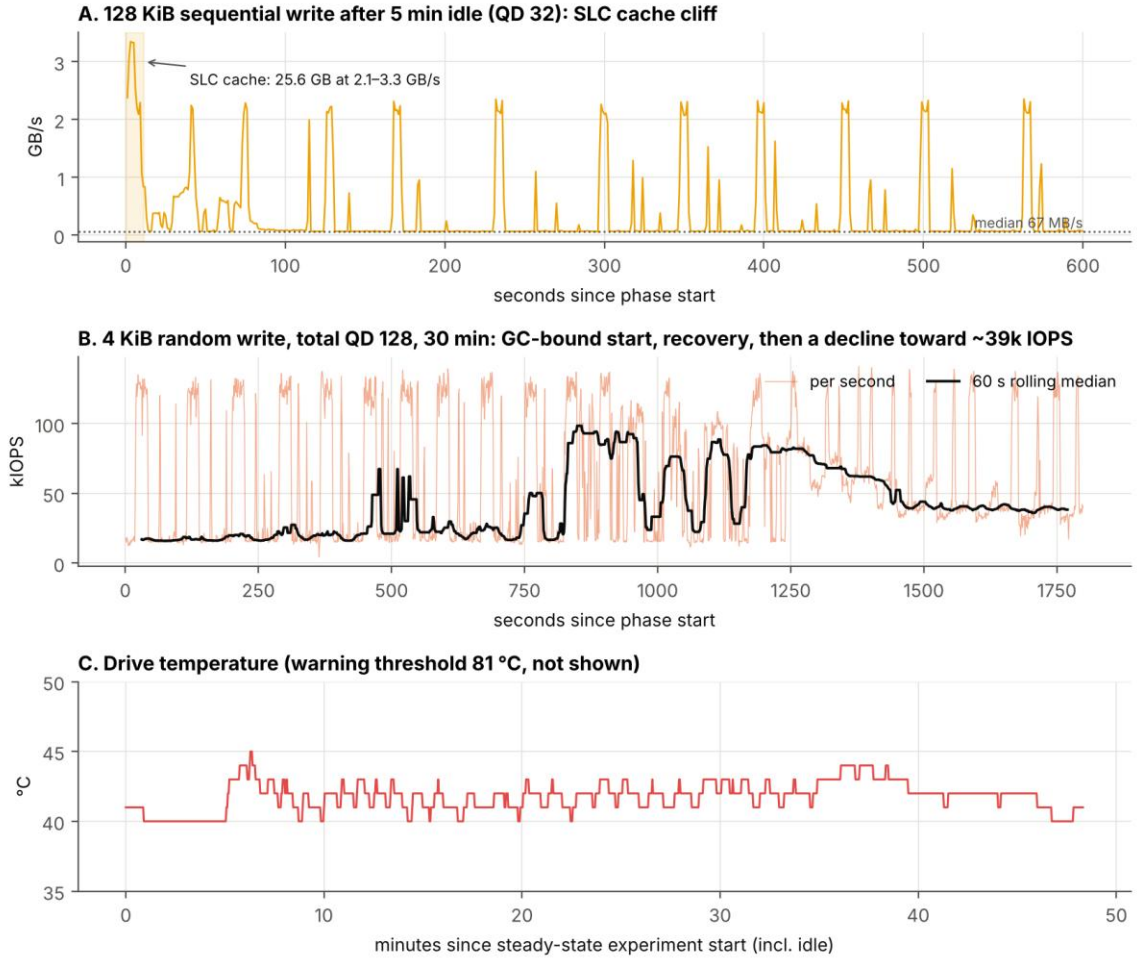


Figure 8: (A) Sequential write throughput per second; (B) random-write IOPS per second with a 60 s rolling median; (C) drive temperature over the whole experiment.

SLC cache cliff. For the first 11 seconds the drive wrote at 2.1–3.3 GB/s, into the part of its TLC flash that it runs in fast single-bit (SLC) mode as a write cache. After 25.6 GB that cache was full, and throughput fell to a median of 67 MB/s for the remaining 589 s: 74% of all seconds sat below 150 MB/s. Every 40–60 s the drive recovered for a few seconds of about 2.15 GB/s (12% of the time), which is the signature of the controller freeing a batch of cache blocks by copying their data into TLC and then accepting new writes at cache speed. Over 600 s the average was 374 MB/s, for 222.8 GB written. The post-cache floor of 67 MB/s is very low for a TLC drive; the likely reason is that the drive was folding cached data into TLC and collecting garbage at the same time on a system disk of unknown fill level (Section 13).

Random-write steady state. Random writes started at the bottom: a median of 19k IOPS for the first 13 minutes, while the drive was still digesting the 223 GB of sequential writes. Between minutes 14 and 21 it recovered to a median of about 76k IOPS, then declined; the final 5 minutes had a median of 39k IOPS with bursts to about 123k. With a coefficient of variation of 0.62 in that last window, the drive had not reached a steady state in the SNIA sense (excursions within ±20% of the average) after 30 minutes. Its sustained level was about 32% of its burst rate, inside the 20–50% I predicted.

Reads after writing. 4 KiB random reads right after the write phase (83,977 IOPS, p99 449 µs) and after 120 s of idle (83,075 IOPS, p99 453 µs) were the same within noise. Background garbage collection, if still running, did not disturb a pure read workload measurably.

Temperature. The drive stayed between 40 and 45 °C during this experiment and peaked at 54 °C during the whole run, far below its 81 °C warning threshold. The throughput drops in panels A and B do not line up with temperature changes, so none of them is thermal throttling. H7 is supported on all three points.

11.3 Write history changes sequential reads

The most surprising device-state effect concerned reads. With identical settings (128 KiB, QD 32, one job), sequential reads ran at 1.87–2.00 GB/s in the block-size sweep but at only 0.97–1.01 GB/s in the queue-depth sweep an hour later.

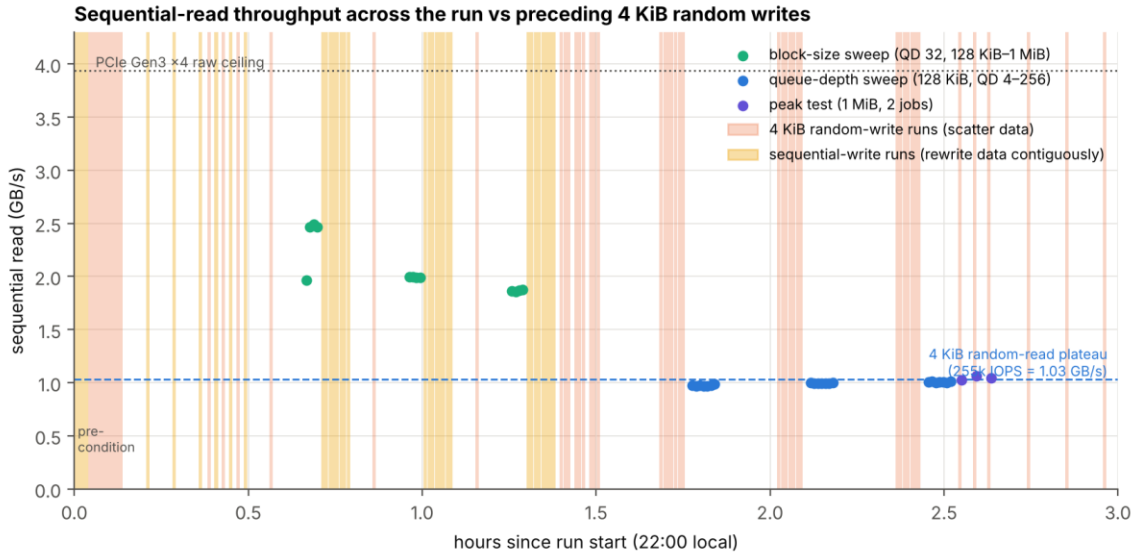


Figure 9: Every sequential-read run of 128 KiB or more at QD ≥ 4, against time of night. Shading marks when 4 KiB random-write runs and sequential-write runs took place.

Temperature does not explain the gap (50–51 °C in both cases). What changed was the write history. Each repetition of the block-size sweep began its reads shortly after sequential writes had rewritten the start of the file in large contiguous pieces. Each repetition of the queue-depth sweep, by contrast, ran its reads directly after about 4.5 minutes of 4 KiB random writes over the whole file, roughly 70 GB per repetition. After that, logically sequential blocks are scattered across the flash at 4 KiB granularity, so a "sequential" 128 KiB read becomes 32 independent 4 KiB flash reads. The plateau supports this precisely: at 0.97–1.04 GB/s it equals the drive's 4 KiB random-read plateau of 255k IOPS × 4 KiB = 1.05

GB/s. Sequential-read throughput is therefore a property of how the data was written, not only of the drive. Reading the file immediately after a fresh sequential rewrite would test this directly (Section 13, anomaly A1).

12. Misleading setups and failure cases

Each case runs a common mistake next to the correct setup and keeps the evidence that exposes it: the volume's byte counters (device bytes ÷ fio bytes) or Little's Law.

Case	Naive setup	Naive result	Correct setup	Correct result	Exposed by
A1 File cache, reads	Buffered 4 KiB reads, 1 GiB, warm	149,847 IOPS at 6.1 µs	Unbuffered	10,939 IOPS at 91.6 µs	Device bytes 0.00 vs 1.08 (13.7× overstatement)
A2 File cache, writes	Buffered 4 KiB writes	144,202 IOPS at 6.2 µs	Unbuffered	32,065 IOPS at 30.1 µs	Device bytes 0.10 vs 1.20 (4.5×)
B1 Sync engine	`sync`, iodepth 32	"QD 32": 9,661 IOPS	`windowsaio`, iodepth 32	83,500 IOPS (reps 1 and 3)	L = 0.99 instead of 32
B2 Jobs × depth	iodepth 32 × 4 jobs, reported as QD 32	260,670 IOPS "at QD 32"	iodepth 32 × 1 job	83,500 IOPS	L = 88.5 instead of ≤ 32
C Never-written blocks	Read an extended, never-written 1 GiB file	209,781 IOPS at 79.6 µs	Read written data	82,479 IOPS at 235 µs	Device bytes 0.000 vs 1.21 (2.5×)
D NTFS zero-fill	16 MiB of 4 KiB random writes into an extended, never-written file	6,621 IOPS, worst write 311–500 ms	Same writes into a fully written file	30,740 IOPS, worst write 2–22 ms	Device bytes 65.0 vs 1.00

Author: Brendan Bianchi

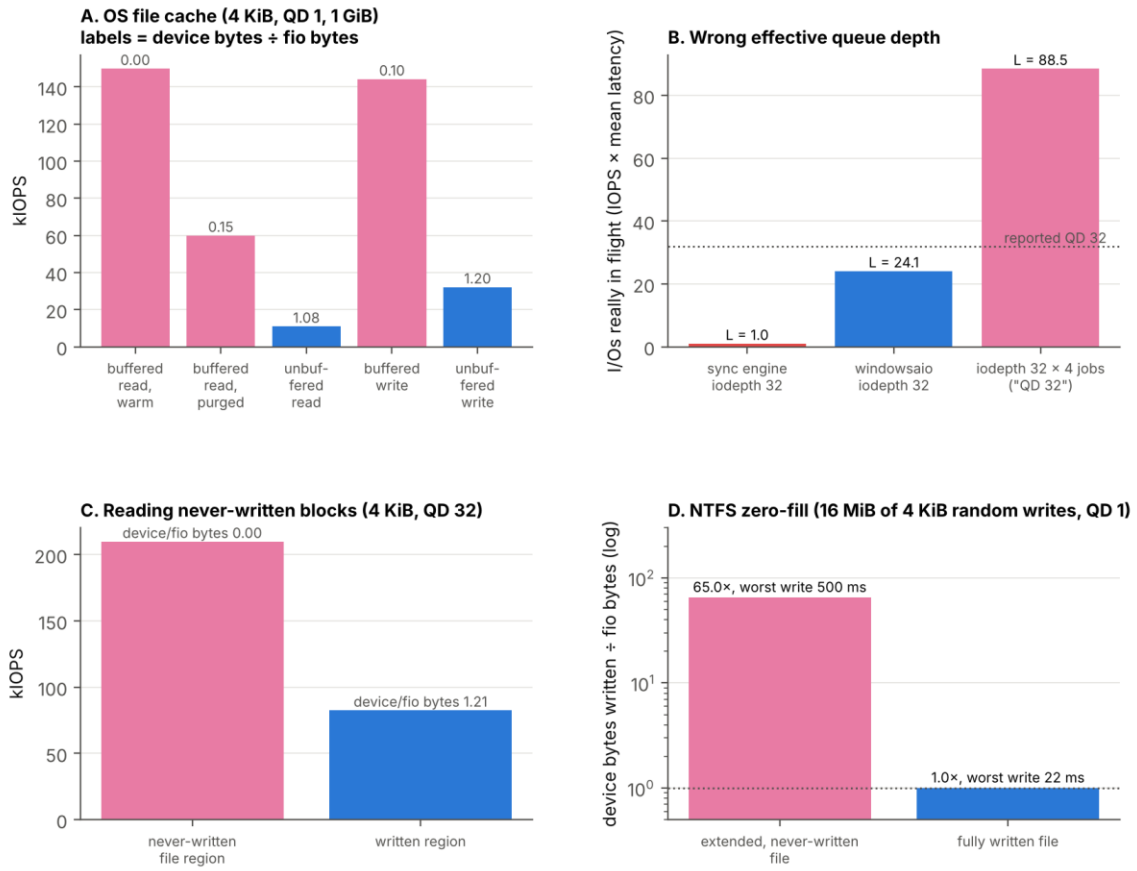


Figure 10: The four misleading setups. Labels give device bytes ÷ fio bytes (A, C), in-flight requests L (B) and the write amplification and worst write latency (D).

A – The Windows file cache. Without `direct=1`, the same 4 KiB random reads over a 1 GiB file reported 150k IOPS at 6 µs, 14 times the drive's real rate, and the volume counters show why: 0.00 bytes were read from the device. The data came from RAM. Buffered writes looked 4.5 times faster than the drive for the same reason: only 10% of the written bytes reached the device during the run, and the rest stayed in the cache to be written later. Even after the harness purged the cache as administrator, the buffered read run measured 60k IOPS, because once the first pass had pulled the 1 GiB working set into RAM (device bytes 0.15, about one working set's worth) every later read was a hit. Any test whose working set fits in RAM measures memory unless it bypasses the cache.

B – Wrong effective queue depth. Windows' synchronous fio engines ignore `iodepth`: with `ioengine=sync` and `iodepth=32`, Little's Law gives L = 0.99, and the 9.7k IOPS equal the QD=1 result. (Windows' synchronous engines also cannot do unbuffered I/O, so this run went through the cache over the full 64 GiB span, where most reads missed.) The opposite mistake, running 4 jobs at iodepth 32 and reporting "QD 32", credited 260k IOPS to a queue depth of 32 while 88.5 requests were really in flight. A reader would wrongly conclude that QD 32 is enough to saturate the drive. Both errors are invisible in fio's output and obvious from IOPS × latency.

C – Never-written blocks. A file whose size was set without writing data reads back as zeros without touching the flash: 210k IOPS from one job, where real data gives 82k, with zero device bytes. On a raw

device the same happens for trimmed or never-written blocks, which the controller answers from its mapping table alone. This is why the harness writes the whole span before any read test.

D – NTFS zero-fill. When a file's end is moved without writing data, NTFS records how far it has really been written (its valid data length). A later write beyond that point must first fill the gap with zeros. Writing 16 MiB of random 4 KiB blocks into a fresh 1 GiB file therefore made the device write 65 times as many bytes, which is the entire 1 GiB file plus the data itself (1 GiB / 16 MiB = 64). Single writes stalled for up to 0.5 s while NTFS zero-filled, and throughput fell 4.6×. A benchmark that creates its test file this way and then measures random writes would report the cost of NTFS, not of the drive. The harness avoids it by letting fio create the file and filling it sequentially first.

Where the technique stops helping

The technique this project studies, more parallelism, has a clear limit. Past the knee, extra queue depth bought nothing: for 128 KiB reads, each doubling from QD 4 to QD 256 doubled latency (to 33.6 ms) at a constant 1.0 GB/s, and for random reads with four jobs, going from QD 64 to 256 lost 2% of throughput while latency tripled. For random writes, deeper queues were actually worse than QD 4. Larger blocks likewise stopped helping at about 32 KiB for reads. In all these cases the only effect of more outstanding work was longer waits.

13. Anomalies, limitations and threats to validity

13.1 Anomalies

#	Observation	Where	Likely explanation (hypothesis)	Possible follow-up test
A1	Identical 128 KiB QD 32 sequential reads ran at 1.87–2.00 GB/s, then at 0.97–1.01 GB/s an hour later	Sec. 11.3	Preceding 4 KiB random writes scattered the data; reads became random 4 KiB flash reads	Rewrite the file sequentially, read at once; repeat after 70 GB of random writes
A2	Random 32 KiB reads slower than 16 KiB (1.05 vs 1.27 GB/s); random ≈ sequential only at 256 KiB	Sec. 6	Same fragmentation at 4 KiB granularity from preconditioning	Repeat the random-read sweep on a sequentially filled file
A3	One job kept only 20 of 32 requests in flight (133 of 256)	Sec. 4, 8	Host-bound: one thread submits and reaps every I/O, at 63% of a core in system time	Batch submission/completion options, more jobs, or the same test under Linux io_uring
A4	Random writes fastest at QD 4 (86k IOPS), slower	Sec. 8	Ordering: QD 4 ran first in each series, right after reads let the	Randomize the QD order or idle before

#	Observation	Where	Likely explanation (hypothesis)	Possible follow-up test
	at every deeper QD		write buffer drain	each point
A5	fio's QD=1 writes 7.5 µs (28%) slower than the reference tool, while reads were 9% faster	Sec. 4	Different completion paths: fio hands each completion from a completion-port thread to the job thread, while the reference waits synchronously	Compare fio's submission and completion latencies; give the reference an overlapped-I/O mode
A6	4 bursty write runs failed the byte check (device ÷ fio = 1.52–1.90)	Sec. 4	The 5 s warm-up ran at cache speed while the measured 30 s were mostly slow, breaking the check's constant-rate assumption	Repeat with zero warm-up or compare per-second logs
A7	About 0.05% of reads took ~2 ms and ~0.001% took 10–80 ms at every queue depth	Sec. 10	Load-independent stalls in the drive (housekeeping, mapping fetch from host memory) or the host	Log per-I/O timestamps and look for periodicity; trace with Windows ETW
A8	One 30 s demo run fell to 5.6k IOPS at QD 32 (mean 5.7 ms) with the queue full (L = 31.7)	Sec. 12, demo B rep 2	A device-side stall, likely internal garbage collection, after a long buffered read pass	Log per second during the demos; re-run that point
A9	After the SLC cache filled, writes fell to a median of 67 MB/s	Sec. 11.2	Folding and garbage collection at once on a system disk of unknown fill level	Record the drive's free space; repeat after TRIM and a long idle
A10	PCIe link at Gen3 ×4 although drive supports more	Sec. 3, 9	Motherboard or slot limited to Gen3 (Ryzen 3000 has Gen4 only on X570/B550 CPU-attached slots)	Check the board model, BIOS PCIe setting and M.2 slot

A1. I expected sequential reads to have one speed for this drive, but the same configuration gave 2.0 GB/s in one experiment and 1.0 GB/s in another, and every repetition agreed within its own experiment. Temperature was the same in both. The explanation in Section 11.3 rests on the coincidence that the low plateau equals the 4 KiB random-read plateau (1.05 GB/s), which is what a fully scattered layout would produce. I am fairly confident in it, but the timing data alone cannot prove it; the follow-up test would settle it in a few minutes.

A2. Random reads of 32 KiB were slower in total throughput than 16 KiB reads, and large random reads caught up with sequential ones only at 256 KiB. After 64 GiB of random 4 KiB preconditioning, a 32 KiB logical block may map to eight scattered flash locations, so its cost grows with size in steps that depend

on the layout. My confidence in the specific 16 vs 32 KiB inversion is low; the repetition ranges overlap little, but I have no independent evidence.

A3. This is the most consequential anomaly, because it limits what the 4 KiB results say about the drive. A single job saturated at 84k IOPS with only 20 of 32 requests outstanding, and four jobs still kept only 61% of a 256-deep queue in flight. fio's latency counts only time from submission to completion, so time a finished request spends waiting to be reaped and resubmitted is invisible in it but shows in Little's Law. The 255k-IOPS plateau is therefore a lower bound on the drive's random-read capability.

A4. In the queue-depth sweep, random writes peaked at the shallowest depth tested. Throughput should never fall as concurrency rises if the device state is constant, so the state was not constant: the QD 4 point always followed several minutes of reads, during which the drive could empty its write buffer. Deeper points ran into a buffer that was already filling. The random-write curve in Figure 4 therefore mixes queue depth with elapsed write volume.

A5. For reads, fio was 9% faster than the reference; for writes, 17–28% slower. A single fixed overhead cannot explain both signs. Writes complete in about 27 µs, so any per-I/O host cost of a few microseconds is a large fraction; reads take about 100 µs, and the reference's synchronous wait appears to add its own wake-up cost there. I treat fio's QD=1 write latency as an upper bound by up to about 7 µs.

A6. The byte-accounting check expects the device to see fio's bytes plus one-sixth for the 5 s warm-up. For writes that started in the fast cache and then slowed, the warm-up carried a much larger share of the bytes. The check failed for the right reason (the rate was not constant), not because data went missing.

A7. The pure-read CCDFs in Figure 6 are identical beyond p99.9 at QD 1, 8 and 32. Since queueing cannot produce a tail that is independent of the queue, these stalls must come from events that would happen anyway. I cannot tell from this data whether they are in the drive or in Windows.

A8. In repetition 2 of demo B, the asynchronous QD 32 run delivered 5.6k IOPS instead of 83.5k, with a mean latency of 5.7 ms. Little's Law gives 31.7 requests in flight, so the queue was full and the delay was inside the drive, not the host. It ran right after a buffered pass over the full 64 GiB, and the drive likely chose that moment for internal work. It affects only the demonstration, whose conclusion ($L \approx 20$–32 for the asynchronous engine) holds in the other two repetitions.

A9. A median of 67 MB/s after the cache filled is far below what TLC flash written directly should sustain. The drive is the system disk and its overall fill level was not recorded. A fuller drive has fewer free blocks, so it must fold cached data and collect garbage at the same time, which fits the periodic recovery bursts in Figure 8A. Without the fill level this remains a hypothesis.

A10. The system report shows the link at 8 GT/s ×4 while the drive advertises up to 32 GT/s. This is not a measurement error, but it halves the drive's potential sequential throughput on this machine, and every interface comparison in this report applies to Gen3.

13.2 Limitations and threats to validity

Several factors bound how far these results generalize. The test partition shares the physical SSD with Windows, so OS activity on C: reached the drive during measurements even though the byte counters

exclude it, and no Microsoft Defender exclusion for the test folder was recorded. The tested span was 64 GiB, 1.7% of the drive, so the drive had far more free space to work with than a full drive would, which favors write results. Preconditioning was not run to a SNIA steady state, and the random-write experiment showed that 30 minutes was not enough to reach one, so write results describe a drive in transition. The Balanced power plan allowed the CPU to drop to 5% minimum state, which affects host-side latency at low load, and Windows cannot isolate CPUs from other work the way Linux can. All 4 KiB results above about QD 16 are partly host-bound (A3). Three repetitions give wide confidence intervals, and for writes the spread mostly reflects the drive's state rather than random noise. Finally, this is one drive in one system on a Gen3 link; the methods carry over, the absolute numbers do not.

14. Conclusions

On this Samsung 990 EVO Plus, a single 4 KiB read costs about 100 µs and a single write about 30 µs, because writes are acknowledged from the drive's buffer while reads must wait for the flash. Throughput comes from parallelism: four jobs raised random reads from 10k to 205k IOPS at the knee (QD 32) for only 50% more latency, and past it, Little's Law held exactly. Every doubling of queue depth then doubled latency without adding throughput, so a latency target, not the drive's maximum, should set the queue depth. Little's Law also proved the most useful validation tool in the project: it showed that a single fio thread could keep only 60% of its configured queue in flight, and it exposed misleading queue depths that fio's own output hides.

The drive's state mattered more than any knob. The SLC cache absorbed 25.6 GB at up to 3.35 GB/s, close to the Gen3 link's limit, after which sustained writes fell to a median of 67 MB/s. Random writes swung between 19k and 123k IOPS depending on how much garbage collection was pending. Even sequential reads halved, from 2.0 to 1.0 GB/s, after heavy random writing scattered the data. Each time, the volume's byte counters or Little's Law exposed a misleading setup that a single number would have hidden: the file cache inflated reads 14×, never-written blocks 2.5×, and NTFS zero-fill multiplied device writes 65×. A natural next step would be to repeat the study in a PCIe 4.0 slot with more submitting threads, recording the drive's fill level, to separate what this platform limits from what the drive itself can do.

Appendix A: Reproducing the results

Run in the "MSYS2 UCRT64" terminal opened as administrator (full guide in `docs/WINDOWS_SETUP.md`):

1. Install the toolchain: `pacman -S make zip unzip git mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-python` plus the numpy, pandas, matplotlib and tabulate packages, and fio from its official Windows installer.
2. Create a dedicated NTFS partition with a `D:\ssdtest` folder; exclude it from Microsoft Defender and from indexing.
3. Write `config.local.env` with `TARGET="D:/ssdtest/fio-scratch.bin"`, `TARGET_SIZE="64G"` and `FIO_CPUS="2-5"`.
4. `make build` — compiles `bin/ref_qd1.exe` and `bin/winstat.exe` with g++.
5. `make smoke` — end-to-end test with tiny sizes, about 5 minutes.
6. `make all` — full run, about 4.5 hours; raw results in `data/raw/<run>/`, tables in `docs/results_tables.md`, harness figures in `plots/<run>/`.

The first attempts surfaced three Windows-specific problems, each fixed in the harness before the measured run. fio was originally given its filename through an environment variable in its escaped form (`D\:/ssdtest/…`), which MSYS2 rewrote as if it were a path list, so fio received a wrong path and failed with "Permission denied". The harness now writes a fully resolved job file for every run, which also documents each run's exact configuration. Native Windows Python ends output lines with `\r\n`, and a stray `\r` broke shell arithmetic in the tail experiment; all captured Python output is now stripped of it. Finally, the harness now refuses to start while an earlier `fio.exe` is still running, and removes a half-written test file from an interrupted run. The figures in this report were drawn from `data/processed/run_20261023_215950/` by a separate plotting script.

Appendix B: Repository layout

Path	Contents
`src/`	g++ tools: `ref_qd1.cpp` (independent QD=1 benchmark), `winstat.cpp` (volume I/O counters, temperature, device info, cache purge, unwritten files)
`fio/`	Job templates: `workload.fio`, `precondition.fio`, `timeseries.fio`, `burst.fio`, `verify.fio`
`scripts/`	`common.sh` (safety checks, run wrapper), `collect_sysinfo.sh`, `win_sysinfo.ps1`, `prepare_target.sh`, `validate.sh`, `exp_*.sh` (one per experiment), `run_all.sh`
`analysis/`	`parse_fio.py`, `validate.py`, `plot.py`, `knee.py`, `fiolib.py`, `sysinfo_summary.py`
`data/raw/<run>/`	Per run: fio JSON, resolved job file and metadata (byte counters, temperature), system info, run log, configuration used
`data/processed/<run>/`	Per-run and summary CSVs, time series, latency histograms, `validation.md`
`docs/`	`REPORT.md` (pre-registered hypotheses), `WINDOWS_SETUP.md`, `results_tables.md`

Appendix C: References

- J. Axboe et al., *fio – Flexible I/O tester* documentation (rev. 3.41): engines, `direct`, Windows filename escaping and the `truncate` zero-fill note.
- Samsung, *990 EVO Plus NVMe M.2 SSD data sheet*: interface, HMB, rated performance and test system.
- Samsung, 990 EVO Plus product page.
- SNIA, *Solid State Storage Performance Test Specification (PTS)*: preconditioning and steady-state definitions.
- Little, J. D. C., "A Proof for the Queuing Formula L = λW," *Operations Research*, 9(3), 1961.

- Kleinrock, L., *Queueing Systems, Volume 2: Computer Applications*, Wiley, 1976 (the "power" criterion used for the knee).